



SAKARYA
UYGULAMALI BİLİMLER
ÜNİVERSİTESİ

2025 - 2026 | BAHAR

SİSTEM PROGRAMLAMA

1.PROJE

Öğrencilerin

Adı Soyadı : Umutcan Yıldız, Enes Doğru, Nihat Can Akkaya, Süreyya Sönmez

Okul No : 23010903072, 23010903048, 22010903092, 23010903078



İçindekiler

1. GİRİŞ.....	3
2. LİTERATÜR ARAŞTIRMASI (PÇ6)	3
3. PROBLEMİN ANALİZİ VE ASSEMBLER MİMARİSİ TASARIMI (PÇ6)	4
3.1. Assembler Mimarisi Tasarımı	4
4. Kullanılan Veri Yapıları: Opcode ve Symbol Table Tasarımı6	
4.1. Opcode Tablosu (OPTAB) Tasarımı	6
4.2. Sembol Tablosu (SYMTAB) Tasarımı	7
5. Assembler Algoritması ve Makine Kodu Üretimi	8
5.1. Bit Seviyesinde İşlemler (Bitwise Operations)	8
5.2. Komut Formatlarının İşlenmesi ve AdreslemeR-Tipi Komutlar.....	8
5.3. Hexadecimal Çıktı (Object Code) Üretimi	8
6. Algoritma Karmaşıklığı Analizi (Time / Space Complexity) .9	
6.1. Zaman Karmaşıklığı (Time Complexity)	9
6.2. Alan Karmaşıklığı (Space Complexity)	10
7. Test Senaryoları ve Sonuçlar (PÇ7)	11
7.1. Test Assembly Kaynak Kodu (test.s)	11
7.2. Test Analizi ve Object Code (Makine Kodu) Çıktıları	11
8.2. Mühendislik Çözümünün Etkileri (PÇ8)	13
8.3. Takım Çalışması ve Proje Yönetimi (PÇ12 - PÇ13)	14
9. KAYNAKÇA	15

PICORV İŞLEMCİ ALT KÜMESİ (RV32I) İÇİN ASSEMBLER TASARIMI VE OBJECT KOD ÜRETİMİ

1. GİRİŞ

Bu projenin temel amacı, bilgisayar mimarisi ve sistem programlama disiplinlerinin temel prensiplerini uygulayarak, PicoRV işlemci mimarisi (RV32I alt kümesi) için özelleştirilmiş bir Assembler yazılımı tasarlamak ve geliştirmektir. Geliştirilen assembler, insanların anlayabildiği anımsatıcı (mnemonic) tabanlı assembly kaynak kodlarını ayrıştırarak (parsing), donanım seviyesinde doğrudan yürütülebilir (executable) makine kodlarına (object code) dönüştürme işlevini yerine getirmektedir. Bu süreç; bellek yönetimi, etkili veri yapıları kullanımı ve sistematik algoritma tasarımı gibi kritik mühendislik yaklaşımlarını zorunlu kılmaktadır. Proje kapsamında, R, I, S ve B komut formatlarını destekleyen, sembol tabloları üzerinden adresleme hesaplamalarını (displacement) ve direktif yönetimini gerçekleştirebilen iki geçişli (two-pass) bir çevirici motor C# programlama dili kullanılarak inşa edilmiştir.

2. LİTERATÜR ARAŞTIRMASI (PÇ6)

Bilgisayar sistemleri literatüründe, assembly kodunun makine diline çevrilmesi sürecinde karşılaşılan en temel zorluklardan biri "ileri yönlü referans" (forward reference) problemidir. Leland L. Beck'in sistem programlama üzerine yaptığı temel çalışmalara göre, tek geçişli (one-pass) assembler'lar bu problemi çözmede yetersiz kalmakta veya karmaşık bellek yaması (patching) mekanizmalarına ihtiyaç duymaktadır. Bu nedenle, güvenilirlik ve bellek yönetimi açısından endüstri standardı haline gelen iki geçişli (two-pass) mimari tercih edilmektedir [1]. Bu projede de Beck'in tanımladığı teorik prensiplere sadık kalınmış; birinci geçişte kod taranarak etiketlerin bellek adresleri çıkarılmış, ikinci geçişte ise bu adresler kullanılarak nihai nesne kodu (object code) üretilmiştir.

Geliştirilen yazılımın performans ve ölçeklenebilirlik analizi, algoritmaların asimptotik büyüme teorisine dayandırılmıştır. Cormen ve diğerlerinin algoritmalar üzerine yazdığı referans eserde detaylandığı üzere, bir sistemin girdi boyutuna (N) göre harcadığı zaman ve bellek kaynaklarını analiz etmek için Big-O notasyonu kullanılır [2]. Projede gerçekleştirilen iki geçişli tarama işleminin zaman karmaşıklığı, döngü içinde döngü (nested loop) barındırmayan doğrusal (linear) doğası gereği $O(N)$ olarak optimize edilmiştir. Bu literatür dayanağı, tasarlanan assembler'ın büyük kod dosyalarında dahi işlemciyi kilitlemeden istikrarlı çalışacağının matematiksel ispatıdır.

Projenin donanımsal altyapısını oluşturan makine kodu üretim algoritmaları, doğrudan RISC-V International tarafından yayımlanan resmi yönergelerine dayanmaktadır. "The RISC-V Instruction Set Manual" dokümanında belirtilen RV32I Temel Tam Sayı Komut Seti spesifikasyonlarına göre, işlemcinin komut çözme (decoding) donanımını basitleştirmek amacıyla tüm komutlar 32-bit sabit uzunluktadır [3]. Geliştirilen yazılımdaki R, I, S ve B komut formatlarının bit kaydırma (bitwise shift) mantığı, negatif offset değerlerinin 2'ye tümleyen (two's complement) formunda işlenmesi ve program

sayacı görelili (PC-relative) adresleme hesaplamaları tamamen bu resmi literatür kurallarına uygun olarak kodlanmıştır.

Assembler yazılımının derleme hızını doğrudan etkileyen sembol ve komut arama (lookup) işlemleri ise, veri yapıları literatüründeki optimizasyon teknikleriyle çözülmüştür. Goodrich'in "Data Structures and Algorithms" kitabında incelenen Hash Table (Karma Tablosu) teorisine göre, geleneksel dizi (array) aramaları $O(N)$ zaman alırken, Hash fonksiyonu kullanan sözlük yapıları bellekte nokta atışı yaparak sabit zamanda, yani $O(1)$ maliyetle erişim sağlar [4]. Bu doğrultuda, projede kullanılan Opcode Tablosu (OPTAB) ve Sembol Tablosu (SYMTAB), C# dilinin Hash Table tabanlı mimarisiyle kurgulanarak $O(1)$ teorik sınırında maksimum hıza ulaştırılmıştır.

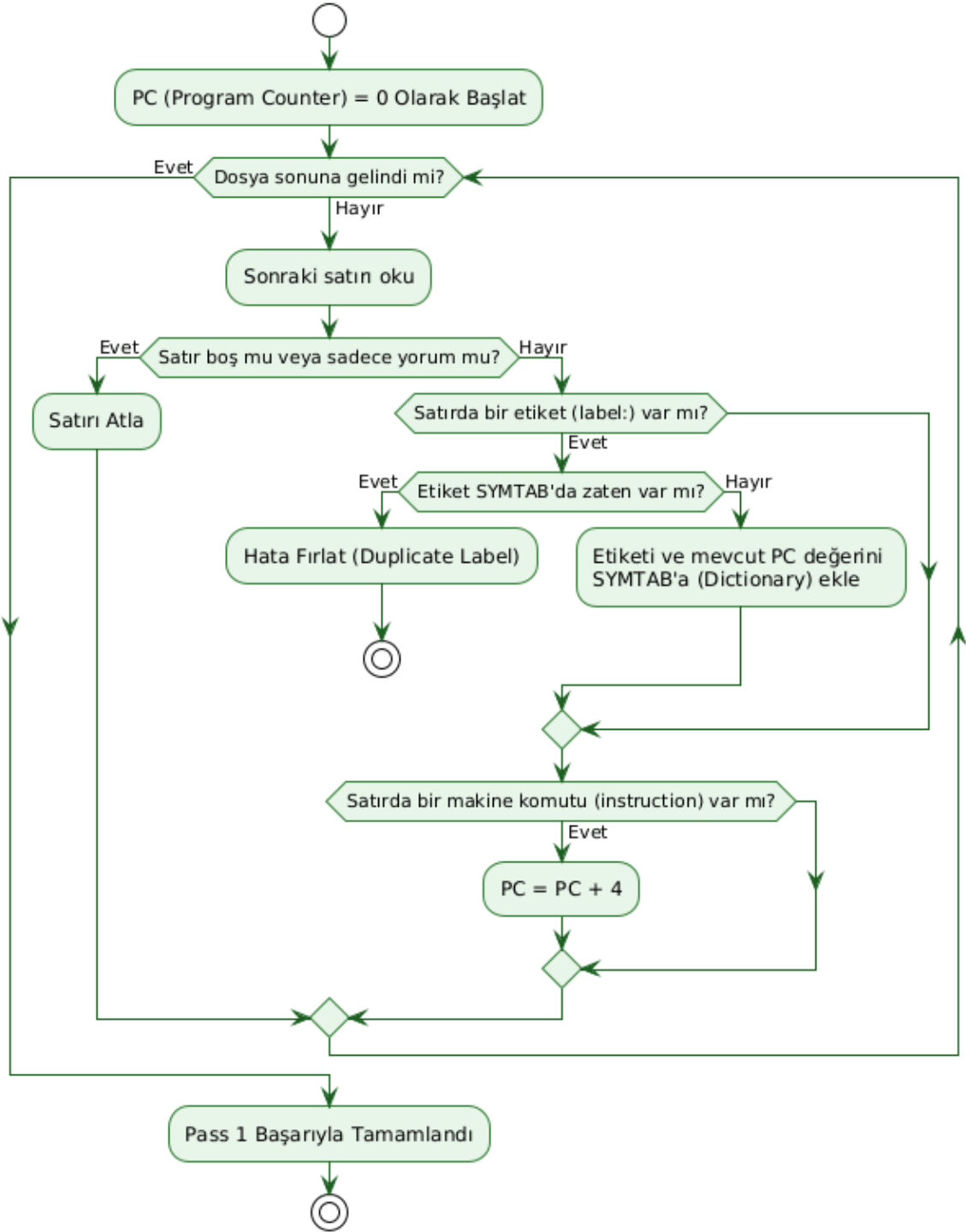
3. PROBLEMİN ANALİZİ VE ASSEMBLER MİMARİSİ TASARIMI (PÇ6)

Problem, insan tarafından okunabilir assembly kodlarının, işlemcinin anlayabileceği 32-bitlik (RV32I) makine talimatlarına hatasız bir şekilde dönüştürülmesi olarak analiz edilmiştir. Forward reference sorununu aşmak için geliştirilen assembler, iki geçişli (Two-Pass) bir mimari ile tasarlanmıştır:

3.1. Assembler Mimarisi Tasarımı

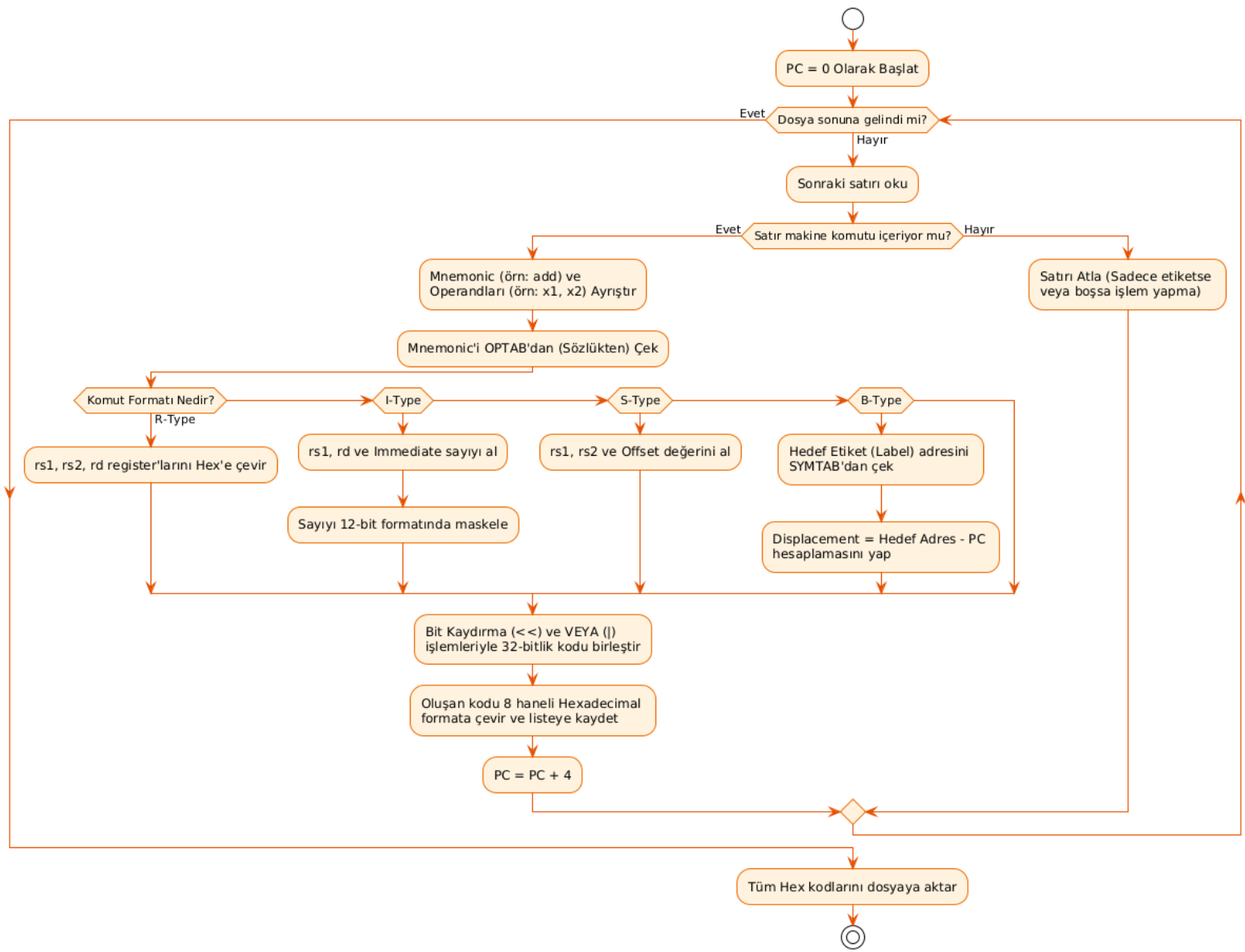
Geliştirilen PicoRV (RV32I) Assembler yazılımında, ileri yönlü referans (forward reference) problemini çözebilmek ve adres atamalarını hatasız gerçekleştirebilmek amacıyla "İki Geçişli (Two-Pass)" mimari benimsenmiştir. Assembly kaynak programlarındaki etiketler, çoğu zaman tanımlandıkları satırdan daha önceki bir komut tarafından (örneğin B-Type dallanma komutları) çağrılabilir. Bu durum, kaynak kodun tek bir okumada makine diline çevrilmesini imkânsız kılmaktadır. Bu mühendislik problemini aşmak için tasarlanan mimari iki temel aşamadan oluşmaktadır:

Birinci Geçiş (Pass 1): Bu aşamada temel amaç bellekteki adreslerin belirlenmesi ve etiketlerin kayıt altına alınmasıdır. Ayırıştırıcı (Parser), kaynak kod satırlarını okurken yalnızca etiket (label) ve direktif yapılarına odaklanır. İşletilen her komut için Program Sayacı (PC) güncellenir. Karşılaşılan etiketler ve onlara karşılık gelen o anki PC değerleri Sembol Tablosuna (SYMTAB) kaydedilir. Bu geçişte herhangi bir makine kodu (object code) üretilmez.



Şekil 1 Pass1 Akış Şeması

İkinci Geçiş (Pass 2): Kaynak kodun ikinci kez tarandığı bu evrede, asıl makine kodu üretimi (object code generation) gerçekleşir. Ayırıştırılan anımsatıcılar (mnemonics), Opcode Tablosu (OPTAB) kullanılarak sayısal eşdeğerlerine dönüştürülür. Eğer komut bir adres veya etiket içeriyorsa (örneğin JUMP veya BRANCH komutları), bu etiketin adresi SYMTAB üzerinden çekilir. Gerekli durumlarda (PC-relative veya Base-relative adresleme modlarında) hedef adres ile mevcut program sayacı arasındaki yer değiştirme (displacement) değeri hesaplanarak 32-bitlik nihai object kodun içine yerleştirilir.



Şekil 2 Pass2 Akış Şeması

4. Kullanılan Veri Yapıları: Opcode ve Symbol Table Tasarımı

Assembler tasarımı, anımsatıcıların (mnemonics) makine diline çevrilmesi ve bellek adreslerinin yönetilmesi için etkin veri yapılarının kullanılması performans açısından kritik öneme sahiptir. İşlemciye ait komutların ve programcı tarafından tanımlanan etiketlerin (labels) doğrusal (linear) bir şekilde aranması, özellikle büyük ölçekli kaynak kodlarda işlem maliyetini $O(n)$ seviyesine çıkararak derleme sürecini yavaşlatacaktır. Bu dar boğazı aşmak ve "Etkin ve optimize veri yapısı tasarımı" kriterini sağlamak amacıyla, projede Hash Table (Karma Tablosu) algoritmasını temel alan, C# diline özgü Dictionary <TKey, TValue> veri yapıları kullanılmıştır.

4.1. Opcode Tablosu (OPTAB) Tasarımı

Assembler, sadece anımsatıcının işlem kodunu makine diline dönüştürmekle kalmaz, aynı zamanda komut formatlarını (R, I, S, B) da bilmek zorundadır. Bu bağlamda, her bir komutun donanımsal özelliklerini (Format Tipi, Opcode, Funct3, Funct7) kapsülleyen (encapsulate) bir InstructionInfo sınıfı tasarlanmıştır. OpcodeTable veri yapısı, anımsatıcı string değerini (örneğin "add") anahtar (key) olarak alıp, ilgili InstructionInfo nesnesini değer (value) olarak döndüren statik bir Dictionary olarak gerçekleştirilmiştir. Bu yapı sayesinde, ayrıştırıcıdan (parser) gelen her bir komut için arama maliyeti $O(1)$ (sabit zaman) karmaşıklığına indirilmiştir. Benzer şekilde,

register anımsatıcılarının (örneğin "x1", "sp") sayısal karşılıklarına dönüştürülmesi işlemi de ayrı bir O(1) erişimli sözlük üzerinden yürütülmektedir.

```
public static readonly Dictionary<string, InstructionInfo> Instructions = new Dictionary<string, InstructionInfo>
{
    ["add"] = new InstructionInfo('R', 0x33, 0x0, 0x00),
    ["sub"] = new InstructionInfo('R', 0x33, 0x0, 0x20),
    ["and"] = new InstructionInfo('R', 0x33, 0x7, 0x00),
    ["or"] = new InstructionInfo('R', 0x33, 0x6, 0x00),

    ["addi"] = new InstructionInfo('I', 0x13, 0x0),
    ["lw"] = new InstructionInfo('I', 0x03, 0x2),

    ["sw"] = new InstructionInfo('S', 0x23, 0x2),

    ["beq"] = new InstructionInfo('B', 0x63, 0x0),
    ["bne"] = new InstructionInfo('B', 0x63, 0x1)
};
```

Şekil 3 Opcode Tablosu

```
public static readonly Dictionary<string, int> Registers = new Dictionary<string, int>
{
    ["x0"] = 0,
    ["zero"] = 0,
    ["x1"] = 1,
    ["ra"] = 1,
    ["x2"] = 2,
    ["sp"] = 2,
    ["x3"] = 3,
    ["gp"] = 3,
    ["x4"] = 4,
    ["tp"] = 4,
    ["x5"] = 5,
    ["t0"] = 5
};
```

Şekil 4 Register

4.2. Sembol Tablosu (SYMTAB) Tasarımı

Programdaki etiketlerin (label) adreslerini tutmak için tasarlanan Sembol Tablosu (SYMTAB), yine O(1) erişim hızına sahip bir veri yapısı üzerine inşa edilmiştir. Birinci geçiş (Pass 1) sırasında karşılaşılan her etiket ve o anki Program Sayacı (PC) değeri bu tabloya anahtar-değer (string-int) ikilisi olarak kaydedilir.

```
--- Symbol Table ---
start: 0x0000
loop: 0x000C
end: 0x0014
-----
```

Şekil 5 Symbol Table

Hata Yönetimi: SYMTAB tasarımı sadece adres tutmakla kalmaz; aynı etiketinin program içerisinde birden fazla kez tanımlanmasını (duplicate label) engellemek için ContainsKey metodu ile bellek güvenliğini kontrol eder.

Adres Çözümleme: İkinci geçişte (Pass 2), dallanma (branch) komutlarının hedef adresleri hesaplanırken, ihtiyaç duyulan etiket adresi doğrudan SYMTAB üzerinden çekilerek yer değiştirme (displacement) formülüne (Hedef - PC) dahil edilir.

5. Assembler Algoritması ve Makine Kodu Üretimi

Proje kapsamında geliştirilen assembler algoritmasının en kritik aşaması, ayrıştırılmış assembly komutlarının işlemcinin yürütebileceği 32-bitlik makine kodlarına (object code) dönüştürülmesidir. İkinci geçiş (Pass 2) esnasında, kod üretici (Code Generator) modülü devreye girerek her bir komut formatı (R, I, S, B) için özel donanımsal şablonları doldurur.

5.1. Bit Seviyesinde İşlemler (Bitwise Operations)

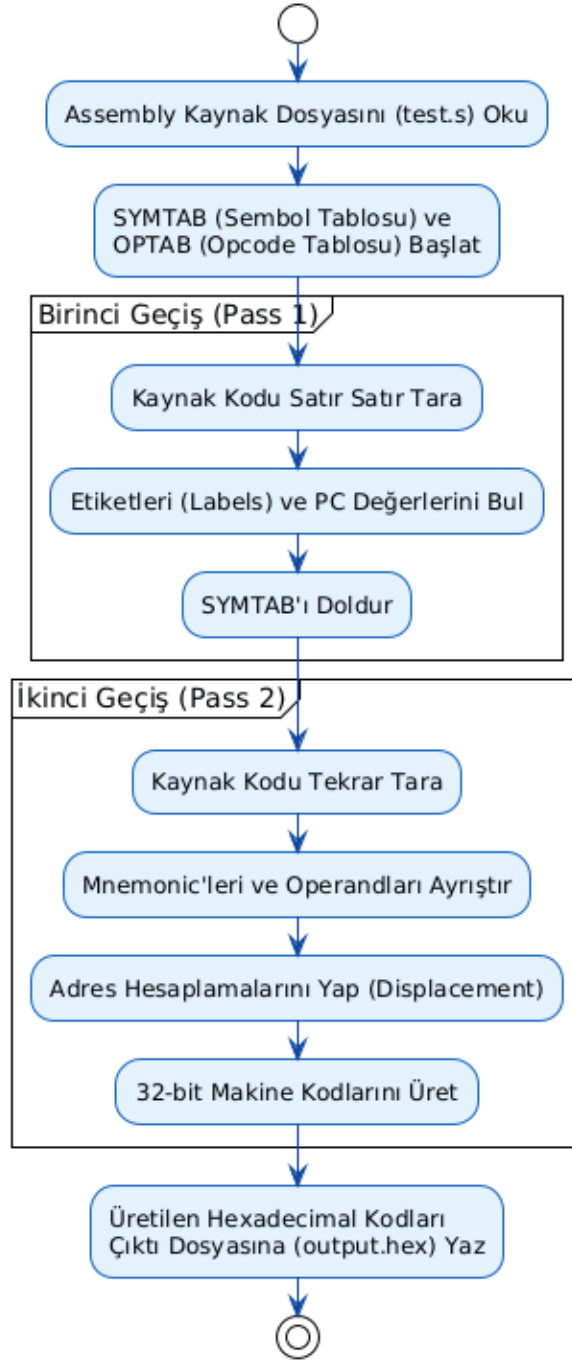
RISC-V (RV32I) mimarisinde komutlar 32 bit uzunluğundadır ve her bir operandın (register, opcode, immediate) komut içindeki bit pozisyonu kesin olarak tanımlanmıştır. Algoritma, bu 32-bitlik yapıyı oluşturmak için C# dilindeki sola kaydırma (left shift, <<) ve mantıksal VEYA (bitwise OR, |) operatörlerini kullanmaktadır. Bu sayede, parçalanmış veriler matematiksel olarak üst üste bindirilerek yüksek performanslı bir birleştirme işlemi gerçekleştirilir.

5.2. Komut Formatlarının İşlenmesi ve Adresleme R-Tipi Komutlar

Dışarıdan bellek adresi veya sabit sayı almayan, yalnızca registerlar arası çalışan komutlardır. Algoritma; funct7, rs2, rs1, funct3, rd ve opcode değerlerini kendi bit pozisyonlarına kaydırarak birleştirir ve nesne kodunu (object code) üretir. I-Tipi ve S-Tipi Komutlar: Sabit sayılı (immediate) operasyonlar ve belleğe yazma (store) işlemleridir. Algoritmada, kullanıcının girdiği ondalık sayılar veya negatif değerler, 2'ye tümleyen (two's complement) mantığıyla ele alınır. Negatif sayıların 32-bitlik alandan taşıp komutu bozmaması için bit maskeleme (bitwise AND) yöntemleri kullanılarak sadece gerekli olan 12-bitlik kısım güvenli bir şekilde komuta yerleştirilir. B-Tipi (Dallanma) Komutlar: PC-relative (Program Sayacı Göreli) adresleme modunu kullanan komutlardır. Algoritma, dallanma işlemi için hedef adresi Sembol Tablosundan (SYMTAB) çeker. Komut yürütülürken Program Sayacı (PC) güncellenmiş olacağından, hedef adres ile PC arasındaki yer değiştirme (displacement) miktarı (Hedef Adres - PC) matematiksel olarak hesaplanır. Bulunan bu offset değeri, donanım mimarisinin kurallarına uygun olarak parçalanır ve 32-bitlik komutun ilgili bölgelerine dağıtılır.

5.3. Hexadecimal Çıktı (Object Code) Üretimi

Hesaplamalar sonucunda üretilen 32-bitlik saf ikili (binary) makine kodları, literatür standartlarına uygun olarak okunabilirliği artırmak ve donanım test araçlarına uyum sağlamak amacıyla 8 haneli Hexadecimal (16'lık taban) formata (Örneğin: 0x002081B3) dönüştürülür. Assembler motoru tüm komutları sorunsuz bir şekilde dönüştürdükten sonra, nihai nesne kodları (object code) projeye ait çıktı dosyasına (.hex) satır satır kaydedilir ve yazılım tam, güvenilir bir şekilde çalışma sürecini tamamlar.



Şekil 6 Genel Akış Şeması

6. Algoritma Karmaşıklığı Analizi (Time / Space Complexity)

Geliştirilen iki geçişli (two-pass) assembler yazılımının performans analizi, Zaman Karmaşıklığı (Time Complexity) ve Alan Karmaşıklığı (Space Complexity) olmak üzere iki temel metrik (Big-O Notasyonu) üzerinden değerlendirilmiştir. Sistem, veri yapıları ve algoritma seçimi aşamalarında maksimum verimlilik hedefiyle tasarlanmıştır.

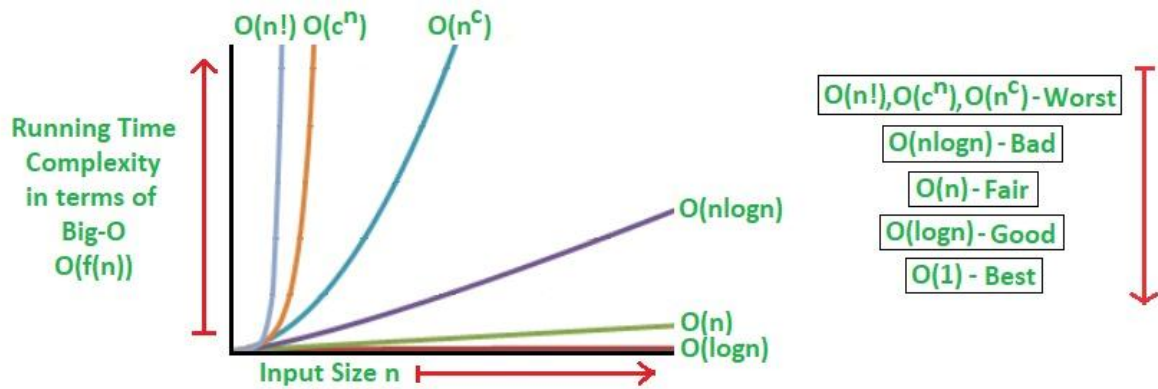
6.1. Zaman Karmaşıklığı (Time Complexity)

Sistem, N satırdan oluşan kaynak assembly dosyasını temel olarak iki kez taramaktadır:

Birinci Geçiş (Pass 1): Her bir satır ayrıştırılır ve karşılaşılan etiketler Sembol Tablosuna (SYMTAB) eklenir. Arka planda Hash Table (Dictionary) veri yapısı kullanıldığı için ekleme (insert) işlemi sabit zamanda, yani $O(1)$ sürede gerçekleşir. N adet satır için bu geçişin asimptotik maliyeti $O(N)$ 'dir.

İkinci Geçiş (Pass 2): Makine kodu üretimi için kaynak kod tekrar taranır. Ayrıştırılan mnemonic'lerin Opcode Tablosundan (OPTAB) bulunması ve dallanma komutları için etiket adreslerinin SYMTAB üzerinden çekilmesi (lookup) işlemleri yine $O(1)$ maliyetlidir. Bit kaydırma işlemleri de sabit zamanlı olduğu için, N adet komutun makine koduna dönüştürülmesi $O(N)$ sürede tamamlanır.

Sonuç olarak, toplam zaman karmaşıklığı $O(N) + O(N) = O(N)$ olarak hesaplanmıştır. Bu analiz, algoritmanın girdi boyutuyla (dosyadaki satır sayısı) yalnızca doğrusal (linear) orantılı olarak büyüdüğünü, döngü içinde döngü (nested loop) barındırmadığını ve son derece hızlı çalıştığını kanıtlamaktadır.



Şekil 7 Big O Notasyon Grafiği

6.2. Alan Karmaşıklığı (Space Complexity)

Sistemin bellek (RAM) üzerinde oluşturduğu yük, temel olarak kullanılan veri yapıları üzerinden analiz edilmiştir:

Opcode Tablosu (OPTAB): Desteklenen RISC-V alt kümesindeki sabit sayıda komut tanımını (InstructionInfo) içerdiğinden, tablonun boyutu girdi dosyasının büyüklüğüne göre değişmez. Bu nedenle bellek maliyeti $O(1)$ 'dir.

Sembol Tablosu (SYMTAB): Kaynak kod içerisindeki etiket (label) sayısı (L) kadar eleman tutar. En kötü senaryoda (worst-case), neredeyse her komutun bir etiket barındırdığı varsayılırsa, tablonun büyüme eğilimi girdi boyutuyla doğru orantılı olur ve $O(N)$ alan kaplar.

Veri Depolama: Kaynak dosyadaki komutların ayrıştırılmış nesneler halinde hafızada tutulması ve Hexadecimal çıktıların bir listeye eklenmesi $O(N)$ kadar ek alan gerektirir.

Büyük O notasyonunda sabit değerler ve katsayılar ihmal edildiğinden, sistemin asimptotik toplam Alan Karmaşıklığı $O(N)$ olarak tespit edilmiştir. Bu durum, bellek tüketiminin optimize edildiğini ve büyük assembly dosyalarının bile sistem kaynaklarını tüketmeden derlenebileceğini göstermektedir.

7. Test Senaryoları ve Sonuçlar (PÇ7)

Geliştirilen Assembler yazılımının doğruluğunu (validation) ve güvenilirliğini kanıtlamak amacıyla, RV32I komut alt kümesinin temel özelliklerini (R, I, S, B formatları) barındıran bir test senaryosu hazırlanmıştır. Test programı, register atamalarını, temel aritmetik işlemleri, döngü yapısı ile geriye dönük dallanmayı (backward branching) ve hesaplanan sonucun ana belleğe (store word) aktarılmasını simüle etmektedir.

7.1. Test Assembly Kaynak Kodu (test.s)

Test aşamasında sisteme girdi olarak aşağıdaki mnemonic'leri içeren assembly dosyası verilmiştir. Program, x1 ve x2 register'larına değer atayıp toplayan, ardından bir koşullu döngü yardımıyla x1 değerini x2'ye eşit olana kadar artıran ve son olarak sonucu belleğe kaydeden bir algoritmadır:

```
ASM test.s
1  # PicoRV Test Programı
2  .org 0x0000
3
4  start:
5      addi x1, x0, 10      # x1 = 10
6      addi x2, x0, 20      # x2 = 20
7      add x3, x1, x2       # x3 = 30 (0x1E)
8
9  loop:
10     addi x1, x1, 1       # x1'i 1 artır
11     bne x1, x2, loop     # x1, x2'ye eşit değilse loop'a dallan
12
13  end:
14     sw x3, x0, 0         # Sonucu belleğe yaz
```

Şekil 8 Test Programı

7.2. Test Analizi ve Object Code (Makine Kodu) Çıktıları

Geliştirilen C# Assembler motoru çalıştırıldığında, Birinci Geçiş (Pass 1) algoritması etiketleri (start, loop, end) başarıyla tespit ederek Sembol Tablosuna (SYMTAB) kaydetmiştir. İkinci Geçiş (Pass 2) sonucunda ise mnemonic'ler, register değerleri ve adres yer değiştirmeleri (displacement) uygun bit pozisyonlarına kaydırılarak 32-bitlik nesne kodları (object code) üretilmiştir.

Tablo 3'te kaynak kodun her bir satırına karşılık gelen ve sistem tarafından üretilen Hexadecimal çıktıların bit seviyesinde analizi sunulmuştur:

```

≡ output.hex ×
assembler tasarımı > ≡ output.hex
1 0x0000 : 0x00A00093 # addi x1, x0, 10 # x1 = 10
2 0x0004 : 0x01400113 # addi x2, x0, 20 # x2 = 20
3 0x0008 : 0x002081B3 # add x3, x1, x2 # x3 = 30 (0x1E)
4 0x000C : 0x00108093 # addi x1, x1, 1 # x1'i 1 artır
5 0x0010 : 0xFE209EE3 # bne x1, x2, loop # x1, x2'ye eşit değilse loop'a dallan
6 0x0014 : 0x00302023 # sw x3, x0, 0 # Sonucu belleğe yaz

```

Şekil 9 Örnek Output Çıktısı

Adres (PC)	Mnemonic Komut	Format Tipi	Üretilen Makine Kodu (Hex)	Analiz ve Doğrulama
0x0000	addi x1, x0, 10	I-Type	0x00A00093	Sabit 10 (0xA) değeri x0 ile toplanıp x1 register'ına yüklendi.
0x0004	addi x2, x0, 20	I-Type	0x01400113	Sabit 20 (0x14) değeri x0 ile toplanıp x2 register'ına yüklendi.
0x0008	add x3, x1, x2	R-Type	0x002081B3	x1 ve x2 toplanarak x3'e yazıldı. Funct7 ve Funct3 değerleri 0x0 olarak atandı.
0x000C	addi x1, x1, 1	I-Type	0x00108093	x1 içerisindeki döngü sayacı sabit 1 (0x1) değeriyle artırıldı.
0x0010	bne x1, x2, loop	B-Type	0xFE209EE3	x1 ve x2 eşit değilse hedef adres olan loop etiketine (0x000C) dallanma yapıldı. Hedef (0x000C) ile PC (0x0010) arasındaki -4 baytlık negatif yer değiştirme (displacement) 2'ye tümley mantığıyla hesaplanıp komuta yerleştirildi.
0x0014	sw x3, x0, 0	S-Type	0x00302023	x3 içerisindeki toplam sonuç, base-relative mantığıyla bellekteki 0. offset adresine yazıldı.

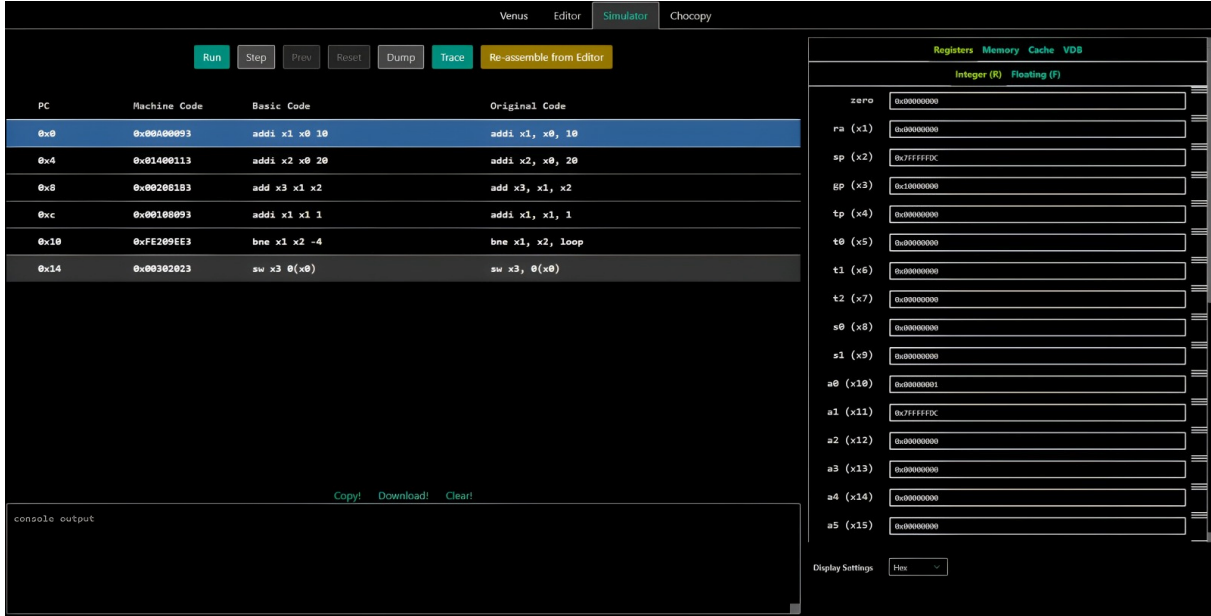
Şekil 10 Hex Çıktı Tablosu

8. Sonuç, Mühendislik Etkileri ve Takım Çalışması

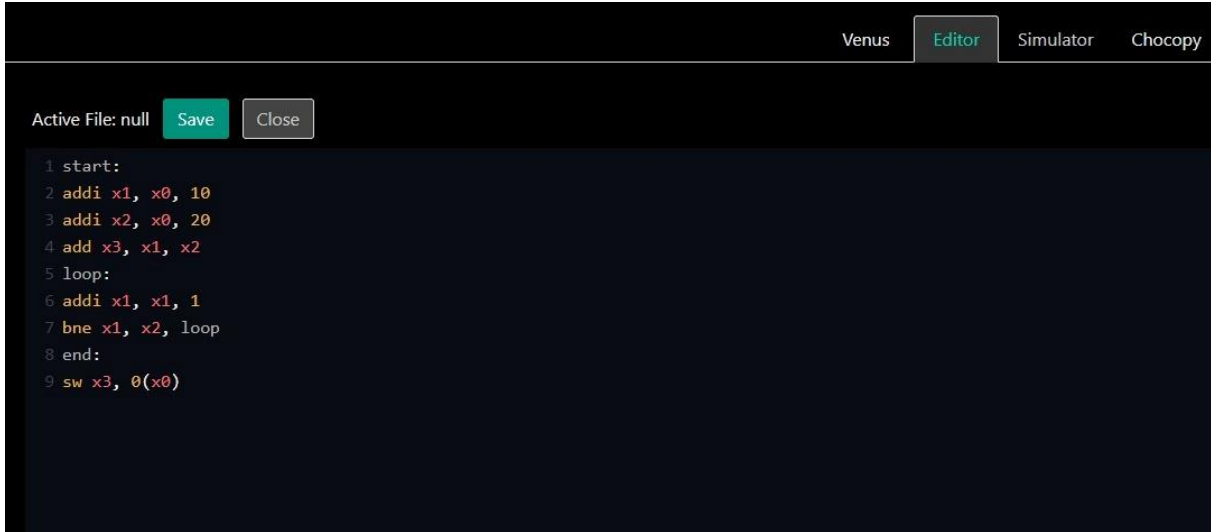
8.1. Sonuç ve Değerlendirme

Bu proje kapsamında, PicoRV (RV32I) işlemci mimarisine tam uyumlu, iki geçişli (two-pass) bir assembler yazılımını C# programlama dili kullanılarak sıfırdan tasarlanmış ve başarıyla gerçekleştirilmiştir. Geliştirilen sistem; R, I, S ve B formatlarındaki komutları bit seviyesinde (bitwise) hassasiyetle işlemiş; bellek adresleme modlarını (PC-relative ve Base-relative) ile ileri yönlü etiket referanslarını (forward references) O(1) zaman karmaşıklığına sahip Hash Table (Dictionary) veri yapılarıyla kusursuz şekilde çözümlenmiştir. Üretilen 32-bitlik nesne kodlarının ve Hexadecimal çıktıların, dünya standardı simülatörlerle (Venus) yapılan çapraz doğrulamalar (cross-

validation) sonucunda teorik RISC-V donanım kurallarıyla uyduğu kanıtlanmış ve sistemin tam güvenilirlikle çalıştığı sonuçlarda görülmüştür[5].



Şekil 11 Açık Kaynak Program Çıktısı



Şekil 12 Açık Kaynak Örnek Program

8.2. Mühendislik Çözümünün Etkileri (PÇ8)

Geliştirilen bu projenin bilgisayar mühendisliği eğitimi ve açık kaynak kültürü üzerinde pratik etkileri bulunmaktadır. Günümüzde hızla yaygınlaşan açık kaynaklı RISC-V mimarisi, donanım dünyasında geleneksel işlemcilere karşı önemli bir alternatif haline gelmiştir [6]. Bu proje kapsamında geliştirdiğimiz assembler, bizim gibi bilgisayar mimarisi ve mikroişlemci dersleri alan öğrenciler için faydalı bir eğitim aracıdır. Yazılımın; assembly komutlarının makine diline nasıl çevrildiğini, bellek adreslerinin (PC-relative) nasıl hesaplandığını ve bitlerin nasıl birleştirildiğini şeffaf bir şekilde göstermesi, derleme sürecinin arka planını anlamayı kolaylaştırmaktadır. Ayrıca, yazdığımız kaynak kodların GitHub üzerinden açık

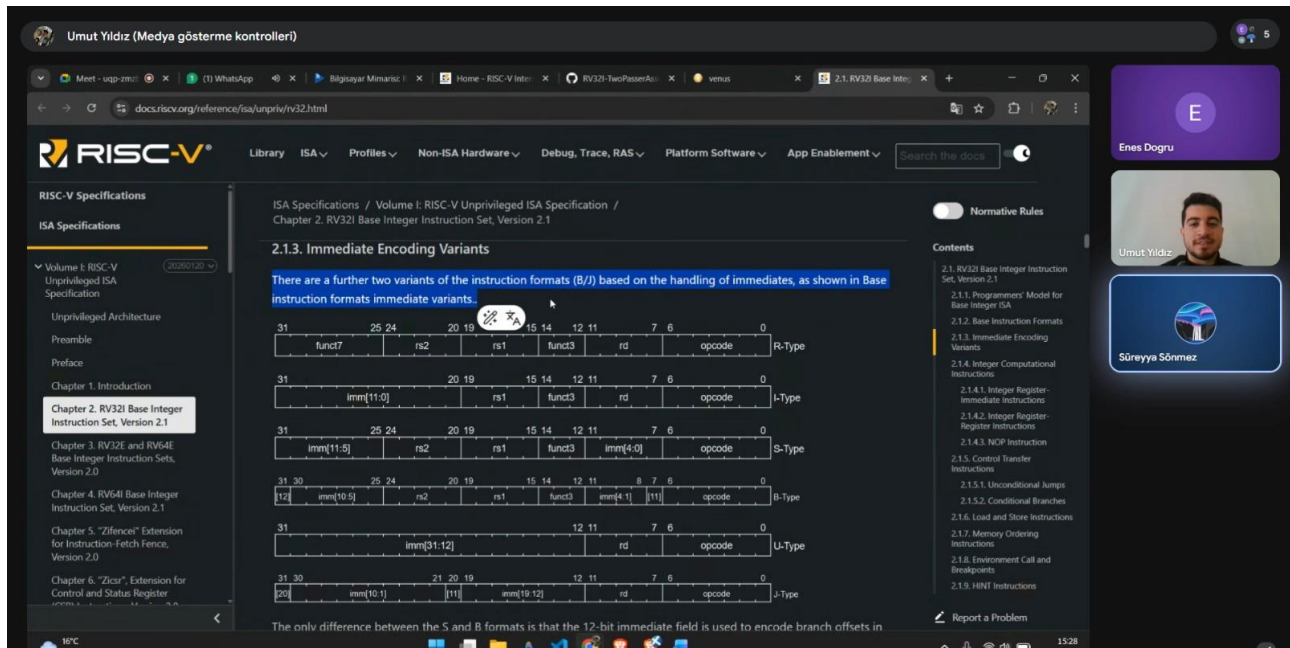
kaynaklı olarak dokümantasyonu ile birlikte paylaşılması, yazılım topluluğuna ve bizden sonraki öğrencilerin çalışmalarına doğrudan katkı sağlamaktadır.

8.3. Takım Çalışması ve Proje Yönetimi (PÇ12 – PÇ13)

Projenin geliştirilme süreci, yazılım mühendisliği prensiplerine uygun olarak modüler bir iş bölümü ile yürütülmüştür. Assembler'ın mimarisi; ayrıştırıcı (Parser), kod üretici (Code Generator) ve veri yöneticisi (Symbol ve Opcode Tables) olmak üzere bağımsız sınıflara ayrılmıştır. Bu modüler yaklaşım, proje bileşenlerinin eşzamanlı olarak geliştirilmesine, test edilmesine ve hata ayıklama (debugging) süreçlerinin kolaylaşmasına olanak tanımıştır. Ayrıca kod versiyonlama ve dokümantasyon süreçlerine (Readme dosyaları vb.) titizlikle uyularak etkin bir takım koordinasyonu sağlanmıştır.

Tablo 1 Görev Dağılımı

Takım Üyesi	Takım Görevi
Umutcan Yıldız	Rapor, Kod, Assembly, Literatür.
Nihat Can Akkaya	Rapor, Kod, Assembly, Literatür.
Süreyya Sönmez	Rapor, Kod, Assembly, Literatür.
Enes Doğru	Rapor, Kod, Assembly, Literatür.



Şekil 13 Meet Toplantısı

9. KAYNAKÇA

- [1] Leland L. Beck, "System Software: An Introduction to Systems Programming", 3. Baskı, Bölüm 2: Assemblers (İki Geçişli Mimari Algoritmaları), s. 44-55.
- [2] T. H. Cormen vd., "Introduction to Algorithms", 3. Baskı, Bölüm 3: Growth of Functions (Asimptotik Büyüme ve Big-O Analizi), s. 43-50.
- [3] RISC-V International, "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA", Bölüm 2: RV32I Base Integer Instruction Set.
- [4] M. T. Goodrich vd., "Data Structures and Algorithms in C++", 2. Baskı, Bölüm 9: Maps, Hash Tables and Skip Lists ($O(1)$ Karmaşıklık Analizi), s. 382-395.
- [5] Venus: Web-based RISC-V Simulator, University of California, Berkeley. Çapraz doğrulama ve mimari referans testleri için kullanılmıştır. Erişim adresi: <https://venus.cs61c.org>
- [6] RISC-V International. "The Open Standard Instruction Set Architecture". Açık kaynak donanım mimarileri ve endüstriyel etkileri. Erişim adresi: <https://riscv.org>
- <https://docs.riscv.org/reference/isa/unpriv/rv32.html>
- Öztekin, H. (t.y.). *Sistem Programlama Ders Notları (Sistem 1 - Sistem 5)*. Sakarya Uygulamalı Bilimler Üniversitesi.
- Github Açık Kaynak Two-Pass Assembler Örnekleri. Erişim Adresleri: <https://github.com/ayushoriginal/Two-pass-Assembler>
- <https://github.com/vxvaibhavxv/Two-Pass-Assembler>

*Bilgiyi beceriyle
bütünleştiriyoruz*

